

LẬP TRÌNH DÀNH CHO KỸ THUẬT

CHƯƠNG 2: KIỂU DỮ LIỆU TUẦN TỰ (PHẦN 1)

ThS. Lê Văn Minh

Giảng viên học phần

— MỤC TIÊU BÀI HỌC

01

Hiểu rõ bản chất của các kiểu dữ liệu phức hợp trong Python và vai trò của chúng trong lập trình kỹ thuật.

03

Làm quen với kiểu danh sách (List) và các phương thức quản lý phần tử linh hoạt.

02

Nắm vững cách sử dụng và các thao tác cơ bản với kiểu chuỗi (String) để xử lý dữ liệu văn bản.

04

Ứng dụng các kiểu dữ liệu này vào giải quyết các bài toán kỹ thuật thực tế một cách hiệu quả.

— TỔNG QUAN KIỂU DỮ LIỆU PHỨC HỢP

Trong kỹ thuật, dữ liệu thường không đơn lẻ mà tồn tại dưới dạng tập hợp các giá trị có liên quan.

Kiểu tuần tự (Sequence)

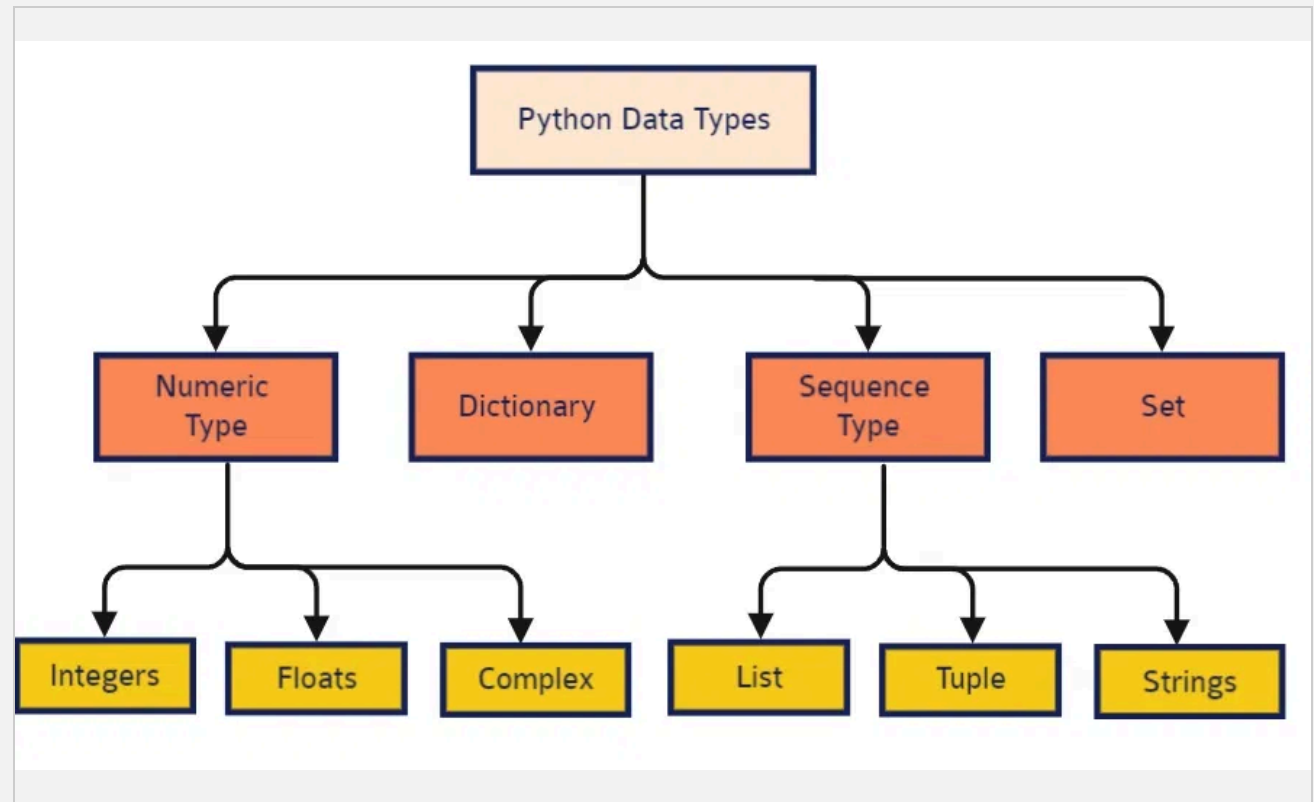
Các phần tử có thứ tự, truy cập qua chỉ mục (String, List, Tuple).

Kiểu ánh xạ (Mapping)

Truy cập qua khóa (Dictionary).

Kiểu tập hợp (Set)

Không trùng lặp, không thứ tự.



— KHÁI NIỆM KIỂU DỮ LIỆU TUẦN TỰ

Kiểu dữ liệu tuần tự (Sequence) là cấu trúc dữ liệu mà các phần tử được sắp xếp và duy trì theo một trật tự nhất định trong suốt vòng đời của nó.

TRUY CẬP VỊ TRÍ

Mỗi phần tử được gắn với một vị trí cụ thể gọi là chỉ mục (index), cho phép truy xuất trực tiếp.

DUYỆT THỨ TỰ

Khả năng duyệt qua các phần tử lần lượt từ đầu đến cuối theo đúng trình tự lưu trữ.

TRÍCH XUẤT ĐOẠN

Hỗ trợ thao tác cắt lát (slicing) để lấy ra một tập hợp con các phần tử liên tiếp.

STR | LIST | TUPLE

— KIỂU DỮ LIỆU STRING (CHUỖI)

Bản chất của chuỗi

String (str) là một dãy các ký tự Unicode được lưu trữ theo một thứ tự xác định từ trái sang phải.

Mỗi ký tự được gán một chỉ mục (index) bắt đầu từ 0.

Là công cụ nền tảng để xử lý văn bản và dữ liệu dạng chữ trong kỹ thuật.

PYTHON EXAMPLE

```
# Khai báo chuỗi  
s = "Python"  
  
# Truy cập ký tự đầu tiên  
print(s[0]) # Kết quả: 'P'
```

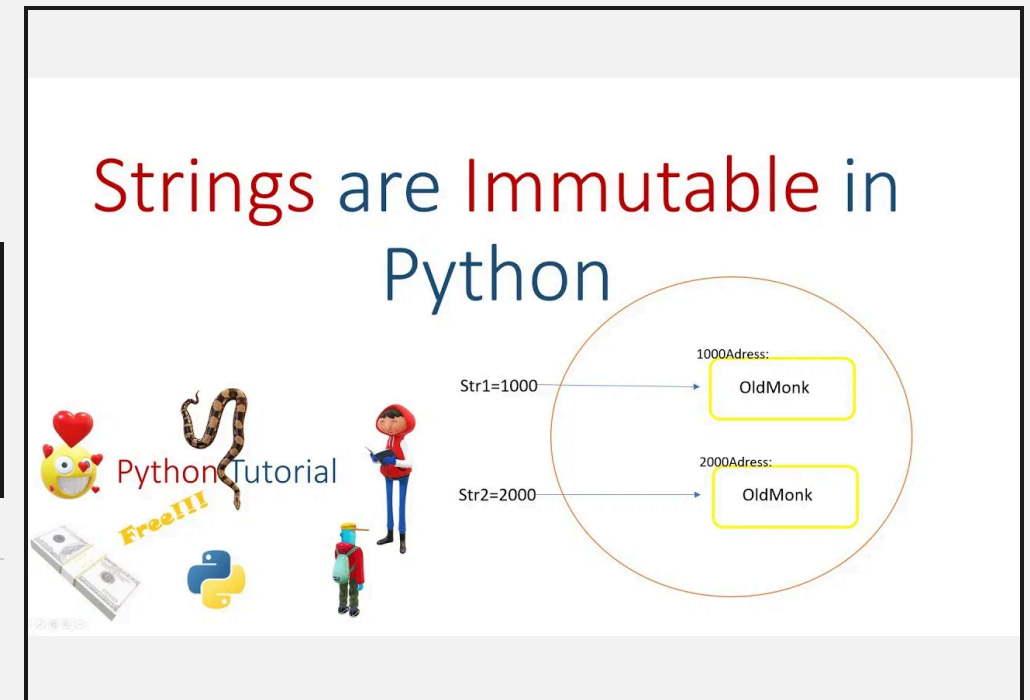
— TÍNH BẤT BIẾN CỦA STRING

KHÁI NIỆM IMMUTABILITY

Sau khi một đối tượng chuỗi được tạo ra trong bộ nhớ, nội dung của nó không thể bị thay đổi trực tiếp.

```
s = "Python"
s[0] = 'p' # Gây lỗi TypeError
TypeError: 'str' object does not support item assignment
```

Ý nghĩa kỹ thuật: Đảm bảo tính toàn vẹn của dữ liệu, giúp chương trình an toàn hơn và tối ưu hóa việc sử dụng bộ nhớ thông qua cơ chế String Interning.



— THAO TÁC CƠ BẢN VỚI STRING

NỐI CHUỖI (+)

Kết hợp hai hoặc nhiều chuỗi lại với nhau để tạo thành một chuỗi mới duy nhất.

```
a = "Lap trinh"  
b = "Python"  
c = a + " " + b # "Lap trinh Python"
```

LẶP CHUỖI (*)

Tạo ra một chuỗi mới bằng cách lặp lại chuỗi ban đầu một số lần xác định.

```
s = "Hi! "  
result = s * 3 # "Hi! Hi! Hi! "
```

KIỂM TRA TỒN TẠI (IN)

Kiểm tra xem một ký tự hoặc chuỗi con có nằm trong chuỗi mẹ hay không (trả về True/False).

```
text = "Python for Engineers"  
print("Python" in text) # True
```

— PHƯƠNG THỨC XỬ LÝ CHUỖI

upper() / lower()

Chuyển đổi toàn bộ chuỗi thành chữ hoa hoặc chữ thường.

strip()

Loại bỏ khoảng trắng thừa hoặc ký tự đặc biệt ở hai đầu chuỗi.

split(separator)

Tách chuỗi thành một danh sách (List) dựa trên ký tự phân cách.

replace(old, new)

Thay thế tất cả các chuỗi con khớp với giá trị cũ bằng giá trị mới.

```
# Ví dụ chuẩn hóa dữ liệu
raw_data = " 25.5 C "
clean_data = raw_data.strip() # "25.5 C"
value, unit = clean_data.split(" ")

# Thay đổi định dạng
msg = "Error: Sensor 01"
print(msg.upper()) # "ERROR: SENSOR 01"
new_msg = msg.replace("01", "02")
```


— KIỂU DỮ LIỆU LIST (DANH SÁCH)

Khái niệm & Đặc điểm

List là cấu trúc dữ liệu tuần tự linh hoạt nhất trong Python, cho phép lưu trữ nhiều phần tử trong cùng một biến.

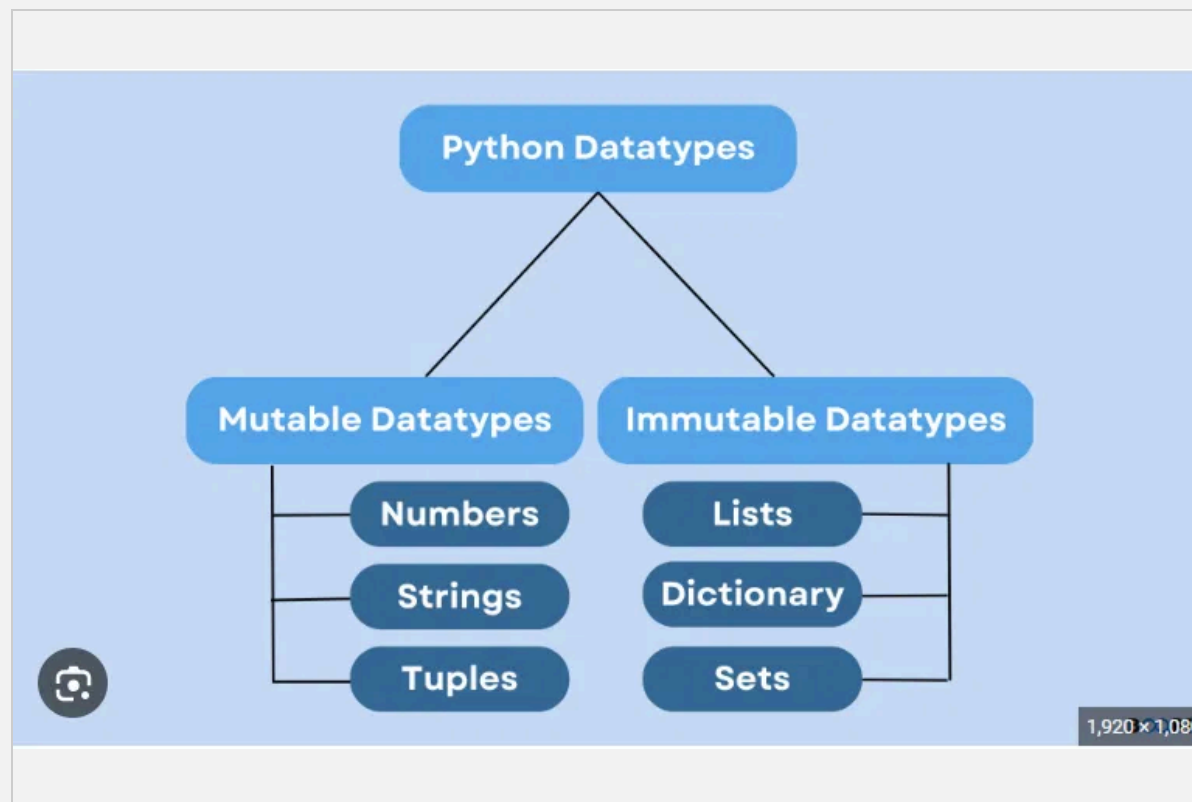
Tính khả biến (Mutable)

Có thể thay đổi, thêm hoặc xóa phần tử sau khi danh sách đã được tạo.

Đa dạng kiểu dữ liệu

Một danh sách có thể chứa các phần tử thuộc nhiều kiểu khác nhau (int, float, str, list...).

```
numbers = [1, 2, 3, 4, 5]
mixed = [1, "Python", 3.14]
```



— TRUY CẬP VÀ CHỈNH SỬA LIST

Tính khả biến (Mutable)

Khác với String, List cho phép thay đổi nội dung của các phần tử trực tiếp sau khi đã khởi tạo.

Sử dụng chỉ mục trong dấu ngoặc vuông [] để xác định vị trí phần tử.

Sử dụng toán tử gán = để cập nhật giá trị mới cho phần tử tại vị trí đó.

```
# Khởi tạo danh sách
data = [10, 20, 30, 40]

# Truy cập phần tử
print(data[1]) # Kết quả: 20

# Chỉnh sửa phần tử
data[0] = 100
print(data) # [100, 20, 30, 40]
```

— THÊM PHẦN TỬ VÀO LIST

Phương thức append()

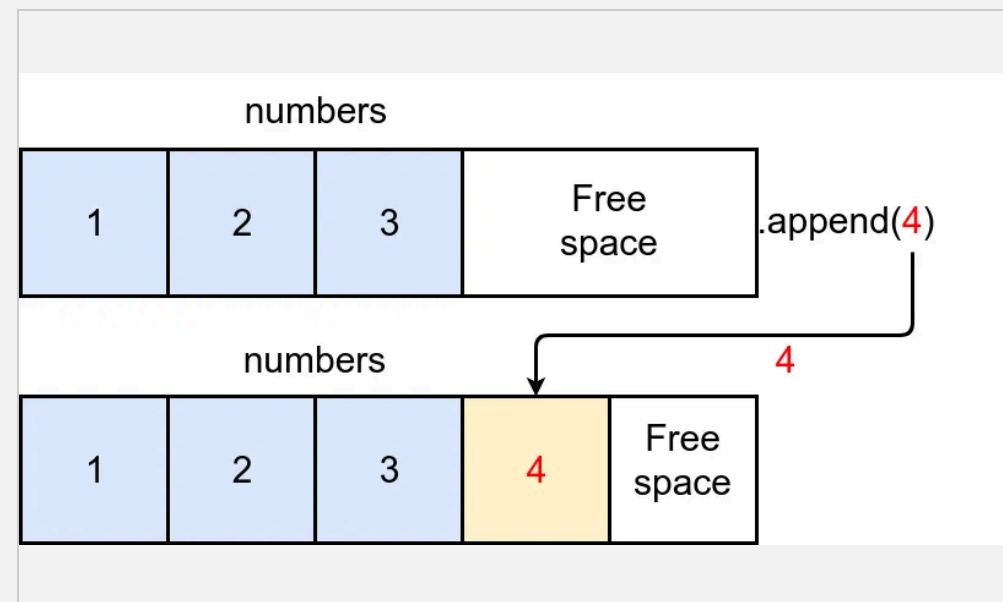
Thêm một phần tử mới vào **cuối** danh sách hiện tại. Đây là thao tác phổ biến nhất để mở rộng List.

```
numbers = [1, 2]
numbers.append(3) # [1, 2, 3]
```

Phương thức insert()

Chèn một phần tử vào một **vị trí bất kỳ** (index). Các phần tử đứng sau vị trí đó sẽ tự động lùi lại.

```
numbers = [1, 2, 3]
numbers.insert(1, 99) # [1, 99, 2, 3]
```



Cơ chế thêm phần tử vào cuối danh sách

— XÓA PHẦN TỬ KHỎI LIST

remove(x)

Xóa phần tử đầu tiên trong danh sách có giá trị bằng x.

```
data = [1, 2, 3, 2]
data.remove(2)
# Kết quả: [1, 3, 2]
```

pop(i)

Xóa và trả về phần tử tại vị trí i. Nếu không ghi i, sẽ xóa phần tử cuối.

```
data = [10, 20, 30]
val = data.pop(1)
# val = 20
# data = [10, 30]
```

del

Câu lệnh hệ thống để xóa phần tử hoặc một đoạn phần tử theo chỉ mục.

```
data = [1, 2, 3, 4, 5]
del data[0]
# data = [2, 3, 4, 5]
del data[1:3]
# data = [2, 5]
```

LẬP TRÌNH DÀNH CHO KỸ THUẬT

CHƯƠNG 2: KIỂU DỮ LIỆU TUẦN TỰ (PHẦN 2)

ThS. Lê Văn Minh

Giảng viên học phần

— KIỂU DỮ LIỆU TUPLE (BỘ DỮ LIỆU)

Khái niệm & Bản chất

Tuple là một cấu trúc dữ liệu tuần tự dùng để lưu trữ một tập hợp các phần tử theo thứ tự xác định, tương tự như List.

Tính bất biến (Immutable): Sau khi được tạo ra, không thể thêm, xóa hoặc thay đổi các phần tử của nó.

Cú pháp: Sử dụng dấu ngoặc đơn () để khai báo.

PYTHON EXAMPLE

```
# Khai báo Tuple
point = (10, 20)

# Tuple hỗn hợp
info = ("ID01", 25.5, "C")
```

— TRUY CẬP PHẦN TỬ TRONG TUPLE

Tương tự như String và List, Tuple sử dụng chỉ mục (index) để truy cập phần tử. Hỗ trợ cả chỉ mục dương (từ đầu) và chỉ mục âm (từ cuối).

```
point = (10, 20, 30)
```

```
# Chỉ mục dương (0 đến n-1)
```

```
x = point[0] # Kết quả: 10
```

```
# Chỉ mục âm (-1 đến -n)
```

```
z = point[-1] # Kết quả: 30
```

— TẠI SAO NÊN SỬ DỤNG TUPLE?

AN TOÀN DỮ LIỆU

Tính **bất biến** giúp bảo vệ dữ liệu khỏi các thay đổi ngoài ý muốn, đảm bảo tính toàn vẹn của các hằng số hoặc tham số cấu hình.

HIỆU NĂNG TỐI ƯU

Tuple có tốc độ truy xuất **nhANH hơn** và tiêu tốn ít bộ nhớ hơn so với List do cấu trúc dữ liệu cố định.

KHÓA DICTIONARY

Vì là kiểu dữ liệu bất biến (hashable), Tuple có thể được dùng làm **khóa (key)** trong Dictionary, điều mà List không thể làm được.

ỨNG DỤNG KỸ THUẬT

Lý tưởng để lưu trữ các nhóm dữ liệu cố định như **tọa độ (x, y)**, thông số cảm biến, hoặc trả về nhiều giá trị từ một hàm.

— KIỂU DỮ LIỆU DICTIONARY (TỪ ĐIỂN)

Bản chất Ánh xạ (Mapping)

Dictionary là kiểu dữ liệu lưu trữ thông tin dưới dạng các cặp **Key:Value** (Khóa:Giá trị).

| Key: Phải là duy nhất và thuộc kiểu dữ liệu bất biến (str, int, tuple).

| Value: Có thể là bất kỳ kiểu dữ liệu nào và có thể trùng lặp.

| Truy cập dữ liệu thông qua Khóa thay vì chỉ mục.

PYTHON DICTIONARY

```
student = {  
    "name": "An",  
    "age": 20,  
    "score": 8.5  
}  
  
# Truy cập qua khóa  
print(student["name"])
```

— TRUY CẬP VÀ THAY ĐỔI DICTIONARY

Thao tác với Key-Value

Dictionary cho phép truy xuất và cập nhật dữ liệu cực kỳ linh hoạt thông qua các khóa (keys).

| Truy cập: Sử dụng `dict[key]`. Lưu ý: Sẽ gây lỗi nếu key không tồn tại.

| Cập nhật: Gán giá trị mới cho một key đã tồn tại.

| Thêm mới: Gán giá trị cho một key chưa có trong Dictionary.

```
student = {"name": "An", "age": 20}

# Truy cập
print(student["name"]) # "An"

# Cập nhật giá trị
student["age"] = 21

# Thêm cặp key-value mới
student["major"] = "Physics"

print(student)
```

— DUYỆT DICTIONARY (ITERATION)

keys()

Trả về một danh sách chứa tất cả các khóa (keys) có trong Dictionary.

values()

Trả về một danh sách chứa tất cả các giá trị (values) tương ứng.

items()

Trả về danh sách các cặp (key, value) dưới dạng Tuple, cực kỳ hữu ích khi duyệt đồng thời.

```
student = {"name": "An", "age": 20}
```

```
# Duyệt qua cả khóa và giá trị  
for key, value in student.items():  
    print(f"{key}: {value}")
```

```
# Kết quả:  
# name: An  
# age: 20
```

— ỨNG DỤNG CỦA DICTIONARY

LƯU TRỮ ĐỐI TƯỢNG

Quản lý thông tin có cấu trúc của các thực thể trong hệ thống kỹ thuật.

Ví dụ: Thông số kỹ thuật của thiết bị, thông tin sinh viên, thuộc tính cảm biến.

TRA CỨU NHANH

Tìm kiếm thông tin tức thời dựa trên một mã định danh duy nhất (Key).

Ví dụ: Tìm tên sinh viên qua MSSV, tìm giá trị đo qua mã cảm biến.

CẤU HÌNH HỆ THỐNG

Lưu trữ các tham số cài đặt của chương trình dưới dạng tên tham số và giá trị.

Ví dụ: Cài đặt cổng kết nối (Port), địa chỉ IP, ngưỡng cảnh báo nhiệt độ.

— KIỂU DỮ LIỆU SET (TẬP HỢP)

Đặc điểm của Set

Set là kiểu dữ liệu dùng để lưu trữ một tập hợp các phần tử không trùng lặp và không có thứ tự xác định.

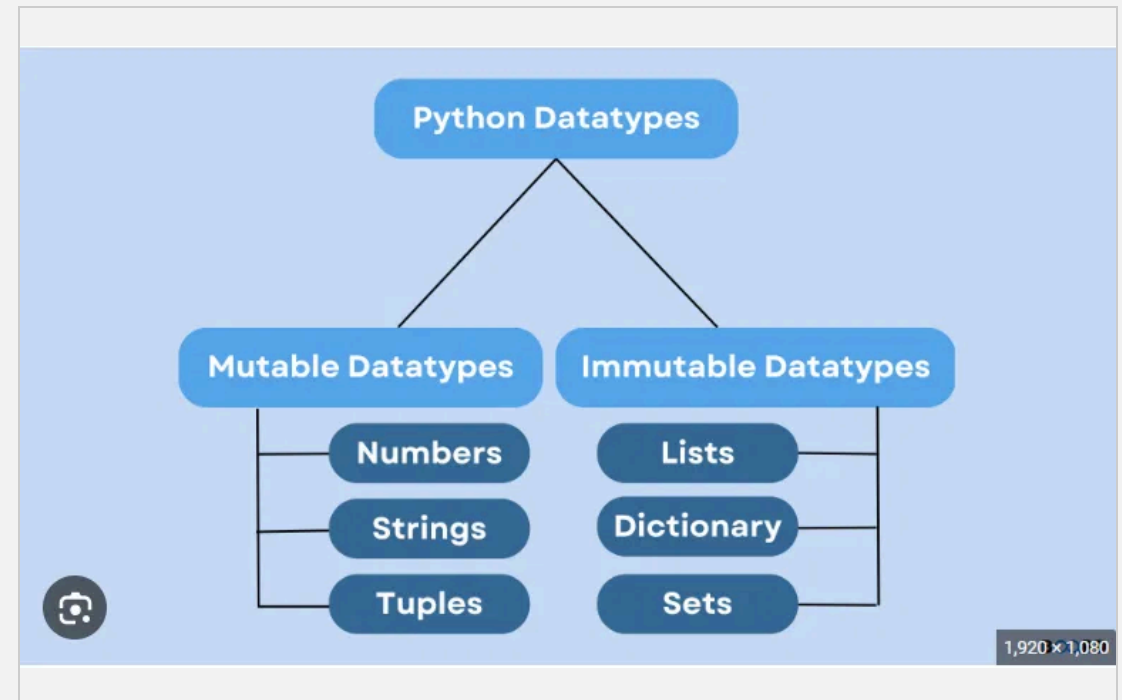
Không trùng lặp

Tự động loại bỏ các phần tử giống nhau. Mỗi giá trị chỉ tồn tại duy nhất một lần.

Không thứ tự

Các phần tử không được duy trì theo trật tự chèn vào và không hỗ trợ truy cập qua chỉ mục.

```
numbers = {1, 2, 3, 4, 4}
print(numbers) # {1, 2, 3, 4}
```



— THAO TÁC CƠ BẢN VỚI SET

Quản lý phần tử

Set cung cấp các phương thức để thêm và xóa phần tử một cách nhanh chóng, nhưng không hỗ trợ truy cập qua chỉ mục.

Thêm phần tử: Sử dụng `add()`. Nếu phần tử đã tồn tại, Set sẽ không thay đổi.

Xóa phần tử: Sử dụng `remove()` (gây lỗi nếu không có) hoặc `discard()` (không gây lỗi).

Lưu ý: Vì không có thứ tự, bạn không thể dùng `[]` để lấy phần tử.

```
numbers = {1, 2, 3}
```

```
# Thêm phần tử
```

```
numbers.add(4)
```

```
numbers.add(2) # Không có tác dụng
```

```
# Xóa phần tử
```

```
numbers.remove(3)
```

```
numbers.discard(10) # An toàn, không  
lỗi
```

```
print(numbers) # {1, 2, 4}
```

— CÁC PHÉP TOÁN TẬP HỢP TRÊN SET

Hợp (Union |)

Lấy tất cả các phần tử duy nhất có trong cả hai tập hợp.

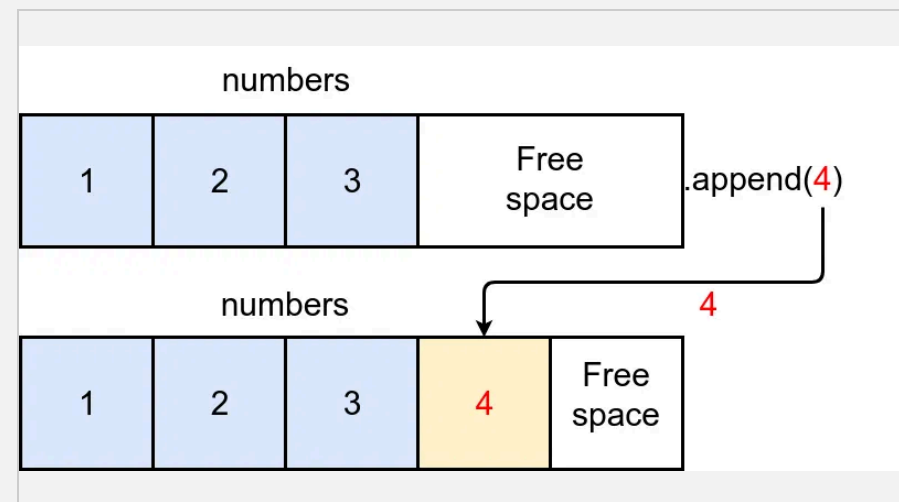
Giao (Intersection &)

Chỉ lấy những phần tử xuất hiện đồng thời ở cả hai tập hợp.

Hiệu (Difference -)

Lấy các phần tử có trong tập hợp A nhưng không có trong tập hợp B.

```
A = {1, 2, 3} ; B = {3, 4, 5}
print(A | B) # {1, 2, 3, 4, 5}
print(A & B) # {3}
print(A - B) # {1, 2}
```



Minh họa các phép toán tập hợp trong Python

— SO SÁNH CÁC KIỂU DỮ LIỆU

ĐẶC ĐIỂM	LIST	TUPLE	SET	DICTIONARY
Thứ tự	Có	Có	Không	Không
Thay đổi	Có (Mutable)	Không (Immutable)	Có	Có
Trùng lặp	Cho phép	Cho phép	Không	Key không trùng
Truy cập	Chỉ mục (Index)	Chỉ mục (Index)	Không hỗ trợ	Khóa (Key)

LẬP TRÌNH DÀNH CHO KỸ THUẬT

CHƯƠNG 2: CẤU TRÚC ĐIỀU KHIỂN & BÀI TẬP (PHẦN 3)

ThS. Lê Văn Minh

Giảng viên học phần

— CẤU TRÚC Rẽ NHÁNH IF-ELSE

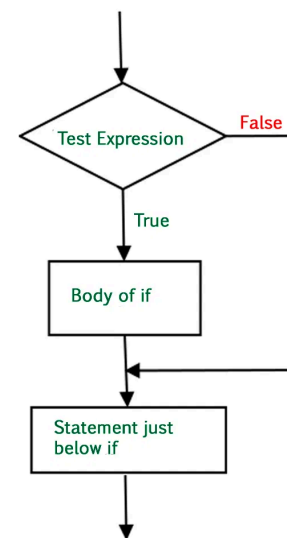
Điều khiển luồng chương trình

Cho phép chương trình thực hiện các khối lệnh khác nhau dựa trên kết quả của một biểu thức điều kiện (True/False).

Thụt lề (Indentation): Python sử dụng khoảng trắng để xác định khối lệnh. Đây là quy tắc bắt buộc.

Cú pháp: Hỗ trợ `if`, `elif` (else if) và `else`.

```
if score ≥ 5:  
    print("Đạt")  
else:  
    print("Không đạt")
```



— VÒNG LẶP FOR

Duyệt qua cấu trúc tuần tự

Vòng lặp for được dùng để duyệt qua các phần tử của một tập hợp (List, Tuple, String, Dictionary) hoặc một dãy số.

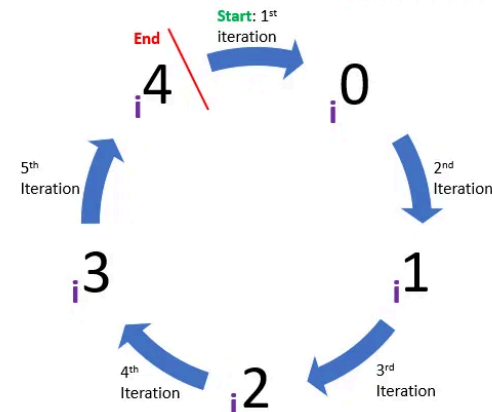
Hàm range(): Tạo ra một dãy số nguyên. Ví dụ range(5) tạo dãy từ 0 đến 4.

Ứng dụng: Thực hiện một khối lệnh với số lần lặp xác định trước.

```
# Lặp với dãy số
for i in range(5):
    print(i) # In 0, 1, 2, 3, 4

# Duyệt qua danh sách
fruits = ["apple", "banana"]
for f in fruits:
    print(f)
```

for i in range(5) → range(5) = Start = 0, Stop = 5, Step = 1



— VÒNG LẶP WHILE

Lặp dựa trên điều kiện

Vòng lặp `while` lặp lại một khối lệnh chừng nào điều kiện logic còn mang giá trị `True`.

Cơ chế: Kiểm tra điều kiện trước khi thực hiện khối lệnh.

Lưu ý quan trọng: Phải có lệnh thay đổi điều kiện bên trong vòng lặp để tránh vòng lặp vô tận.

```
count = 0
while count < 5:
    print(count)
    count += 1 # Tăng biến đếm

# Kết quả: In từ 0 đến 4
```

— BÀI TẬP THỰC HÀNH TẠI LỚP

BÀI TẬP 1

Viết chương trình nhập vào một **danh sách (List)** điểm số của sinh viên. Tính điểm trung bình của cả lớp và in kết quả ra màn hình.

BÀI TẬP 2

Sử dụng **Dictionary** để lưu trữ thông tin 3 sinh viên (Tên, MSSV, Điểm). Viết vòng lặp in ra thông tin các sinh viên có **điểm > 5**.

BÀI TẬP 3

Viết chương trình tìm giá trị lớn nhất trong một **Tuple** số nguyên cho trước bằng cách sử dụng vòng lặp và cấu trúc điều kiện.

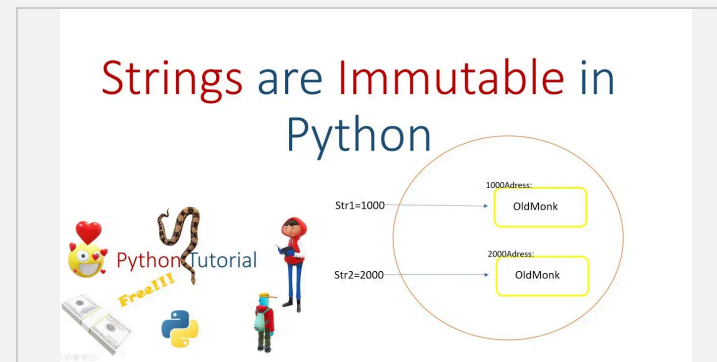
— TỔNG KẾT & HƯỚNG DẪN TỰ HỌC

Kiến thức trọng tâm

- Nắm vững 4 kiểu dữ liệu tuần tự: **List, Tuple, Dictionary, Set**.
- Phân biệt rõ tính **Khả biến (Mutable)** và **Bất biến (Immutable)** của các kiểu dữ liệu.
- Sử dụng thành thạo các cấu trúc điều khiển: **if-else, vòng lặp for, vòng lặp while**.
- Kỹ năng truy xuất, thêm, xóa và duyệt qua các tập hợp dữ liệu.

Yêu cầu tự học

- Hoàn thành các bài tập thực hành trong giáo trình Chương 2.
- Cài đặt và làm quen với môi trường **VS Code / PyCharm**.
- Chuẩn bị nội dung Chương 3: **Hàm và Module**.



Hệ thống hóa kiểu dữ liệu Python